The 4 most important are:

Trigger.new
Trigger.old
Trigger.newMap
Trigger.oldMap

Trigger.new

Contains the new/current version of records.

```
for (Customer__c customer : Trigger.new) {
    System.debug(customer.Name);
}
```

Example during update:

Before: Status = Inactive
User changes it
After: Status = Active

Trigger.new → Active

Available in:

Insert ✅
Update ✅
Undelete ✅
Delete ❌

Trigger.old

Contains the previous/old version of records.

Example:

Inactive → Active

Trigger.old → Inactive
Trigger.new → Active

Example code:

```
for (Customer__c customer : Trigger.old) {
    System.debug(customer.Name);
}
```

We used `Trigger.old` in our delete trigger, because after deleting there isn't a new version of that Customer.

DELETE
↓
Trigger.old ✅
Trigger.new ❌

`Trigger.oldMap:`

```
Customer__c oldCustomer =
    Trigger.oldMap.get(customer.Id);
```

A Map works like:

ID → OLD Record

C001 → Krishna (Inactive)
C002 → Rahul (Active)

So:

Trigger.oldMap.get(customer.Id)

means:

> Give me the **old version of this particular Customer**.

That's how we compared:

```
if (
    oldCustomer.Membership_Status__c == 'Inactive'
    &&
    customer.Membership_Status__c == 'Active'
) {
    System.debug('Membership activated');
}
```

. `Trigger.newMap`

Same concept, but contains the **new versions** indexed by ID.

ID → NEW Record

C001 → Krishna (Active)
C002 → Rahul (Inactive)

You can get a specific new Customer:

```
Customer__c newCustomer =
   Trigger.newMap.get(customerId);
```

List vs Map:

Trigger.new
  ↓
LIST of NEW records

Trigger.old
  ↓
LIST of OLD records

Trigger.newMap
  ↓
ID → NEW record

Trigger.oldMap
  ↓
ID → OLD record